

## Chapter10

# 딥시크-R1(DeepSeek-R1) 기반 로컬 LLM 구축 및 활용

## 1. 서론: 로컬 AI 패러다임의 전환과 데이터 주권

### 1.1 클라우드 기반 AI 의 한계와 로컬 대안의 부상

#### 클라우드 AI의 보이지 않는 비용: 데이터, 비용, 그리고 통제권

현재의 클라우드 기반 LLM은 혁신적이지만, 세 가지 핵심적인 리스크를 내포하고 있습니다.



##### 데이터 프라이버시 리스크

모든 데이터를 외부 서버로 전송해야 하므로 금융, 의료, 법률 등 민감 정보의 유출 위험이 상존합니다. 이는 LLM 도입의 가장 큰 장벽입니다. (Source 1)



##### 예측 불가능한 비용

API 호출당 과금되는 토큰 기반 비용 구조는 서비스가 확장될수록 기하급수적으로 증가하여, 지속 가능한 비즈니스 모델 구축을 어렵게 만듭니다. (Source 1)



##### 벤더 종속성(Vendor Lock-in)

특정 기업의 API 정책, 가격 변동, 서비스 중단에 비즈니스가 직접적인 영향을 받게 되어 기술적 통제권을 상실하게 됩니다. (Source 8)

인공지능(AI), 특히 거대 언어 모델(LLM)의 발전은 OpenAI의 GPT 시리즈가 API 형태로 공개되면서 산업 전반에 혁명적인 변화를 가져왔다. 그러나 이러한 SaaS(Software as a Service) 형태의 클라우드 AI 모델은 기업과 개발자들에게 명확한 진입 장벽과 리스크를 안겨주었다. 가장 큰 문제는 데이터 프라이버시와 보안이다. 클라우드 모델을 활용하기 위해서는 필연적으로 내부 데이터가 외부 서버로 전송되어야 하며, 이는 금융, 의료, 법률 등 기밀 정보의 보호가 필수적인 산업군에서 LLM 도입을 저해하는 주된 요인으로 작용한다. 또한, API 호출당 과금되는 토큰(Token) 기반의 비용 구조는 서비스의 확장성에 제동을 건다. 사용자가 늘어날수록 비용이 선형적 혹은 기하급수적으로 증가하는 구조는 지속 가능한 비즈니스 모델 구축에 어려움을 준다.

이러한 배경 속에서 로컬 환경(On-Premise)이나 엣지 디바이스에서 직접 구동 가능한 소규모 언어 모델(sLLM, Small Language Model)에 대한 수요가 폭발적으로 증가하였다. 로컬 LLM은 데이터가 외부로 유출될 염려가 없는 완전한 보안성을 제공하며, 초기 하드웨어 구축 비용을 제외하면

추가적인 추론(Inference) 비용이 발생하지 않는다는 경제적 이점을 제공한다. 특히 인터넷 연결이 제한된 환경이나 보안이 엄격한 내부망에서도 독립적으로 작동할 수 있다는 점은 국방 및 보안 시설에서의 활용 가능성을 시사한다.

## 1.2 딥시크-R1(DeepSeek-R1)의 등장과 기술적 충격

### 패러다임의 전환: GPT-4o급 추론 능력, MIT 라이선스로 완전 개방

2024년 말, 딥시크(DeepSeek)가 공개한 DeepSeek-R1은 '고성능 AI는 빅테크의 전유물'이라는 고정관념을 깨뜨렸습니다.

- **성능의 동등성:** 추론(Reasoning) 능력에서 GPT-4o와 필적하거나 특정 영역(수학, 코딩)에서는 능가하는 성능을 입증했습니다. ([Source 1](#))
- **완전한 개방성:** 가장 허용적인 라이선스 중 하나인 MIT 라이선스로 공개되어, 누구나 상업적 용도를 포함하여 자유롭게 검증하고 활용할 수 있습니다. ([Source 1](#))
- **사고 과정의 투명성:** AI의 '블랙박스'를 열었습니다. `<think>` 태그를 통해 답변 도출까지의 모든 논리적 과정을 투명하게 공개하여, 사용자가 AI의 판단 근거를 직접 검증할 수 있습니다. ([Source 1](#))

R1

```
<think>
사용자가 피보나치 수열 함수를 요청했다.
1. 재귀적 방법은 코드는 간단하지만 성능이 느리다.
2. 반복문을 사용하는 방법이 효율적이다.
가장 일반적인 반복문 방식을 제시하는 것이 좋겠다.
</think>
알겠습니다. 피보나치 수열을 구하는 Python 함수입니다...
```

2024년 말에서 2025년 초, 중국의 AI 기업 딥시크(DeepSeek)가 공개한 딥시크-R1 모델은 전 세계 AI 연구 및 개발 커뮤니티에 큰 충격을 주었다. 기존의 오픈소스 모델들이 상용 모델 대비 성능적 열세를 보였던 것과 달리, 딥시크-R1은 추론(Reasoning) 능력에서 GPT-4o나 OpenAI o1 모델에 필적하거나 특정 영역에서는 능가하는 성능을 보여주었기 때문이다. 딥시크는 "대형 언어 모델은 글로벌 빅테크 기업의 전유물"이라는 고정관념을 깨뜨렸으며, 고성능 모델을 MIT 라이선스라는 매우 개방적인 조건으로 공개하여 누구나 기술을 검증하고 활용할 수 있게 했다.

딥시크-R1의 혁신성은 단순히 성능에 그치지 않는다. 이 모델은 질문에 즉답하는 기존 방식에서 벗어나, 인간의 사고 과정과 유사한 '사고 사슬(Chain of Thought, CoT)'을 내재화하였다. `<think>` 태그를 통해 모델이 답변을 도출하기까지의 논리적 추론 과정을 투명하게 공개함으로써, 사용자는 AI의 판단 근거를 검증할 수 있게 되었다. 또한, 거대 모델의 지식을 작은 모델로 이식하는 증류(Distillation) 기술을 통해, 일반 소비자용 PC에서도 고성능 AI를 구동할 수 있는 길을 열었다. 이러한 딥시크-R1 모델을 심층 분석하고, 이를 로컬 환경에 구축하여 실질적인 애플리케이션을 개발하는 전 과정을 상세히 다룬다.

## 2. 딥시크-R1(DeepSeek-R1) 아키텍처 및 학습 방법론 심층 분석

### 2.1 MoE(Mixture of Experts) 아키텍처와 효율성

#### 더 똑똑하게, 더 효율적으로: MoE 아키텍처의 비밀

DeepSeek-R1은 거대한 모델의 지식은 유지하면서 연산 비용은 획기적으로 줄이는 MoE(Mixture of Experts) 아키텍처를 채택했습니다.

총 파라미터: 6,710억 (671B)

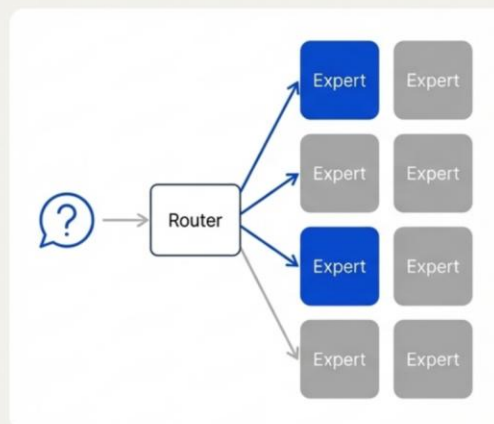
추론 시 활성 파라미터: 약 370억 (37B) - 전체의 **5.5%**만 사용 (Source 2)

##### 작동 원리

질문의 성격에 따라 필요한 '전문가(Expert)' 네트워크 선별적으로 활성화하고 나머지는 유휴 상태로 둡니다. 이는 모든 전문가가 항상 참여하는 회의 대신, 안전에 맞는 전문가만 소집하는 것과 같습니다.

##### 결과

GPT-4o 대비 추론 비용이 약 **1/20** 수준으로 절감되었으며, 동일 하드웨어에서 더 빠른 응답 속도를 제공합니다. (Source 2)



필요한 전문가만 활성화하여 연산 효율 극대화

딥시크-R1의 핵심 경쟁력은 아키텍처의 효율성에서 기인한다. 딥시크-R1(671B 버전)은 총 6,710억 개의 파라미터를 보유하고 있지만, 모든 추론 과정에서 전체 파라미터를 사용하지 않는다. 대신 **MoE(Mixture of Experts)** 구조를 채택하여, 입력된 토큰의 성격에 따라 필요한 '전문가(Expert)' 네트워크만을 선별적으로 활성화한다.

MoE 아키텍처는 거대 모델의 지식 용량을 유지하면서도 연산 비용을 획기적으로 줄이는 기술이다. 실제 딥시크-R1이 추론 시 활성화하는 파라미터(Active Parameters)는 약 370억(37B) 개 수준으로, 이는 전체 파라미터의 약 5.5%에 불과하다. 이러한 구조 덕분에 GPT-4o 대비 추론 비용이 약 1/20 수준으로 절감되었으며, 동일한 하드웨어 자원에서 더 빠른 응답 속도를 제공한다.

## 2.2 순수 강화학습(Pure RL)과 GRPO 알고리즘

### 스스로 추론 능력을 터득하다: 순수 강화학습과 GRPO

DeepSeek-R1은 지도학습(SFT) 의존도를 최소화하고, 강화학습(RL)을 통해 스스로 문제 해결 능력을 극대화했습니다.

- **창발적 행동 (Emergent Behavior):** 초기 모델(DeepSeek-R1-Zero)은 SFT 없이 수만 번의 자체 문제 풀이와 검증을 반복했습니다. 이 과정에서 '자기 검증', '반성', '긴 사고 사슬'과 같은 고도의 지능적 행동이 별도의 지도 없이 자연스럽게 나타났습니다. ([Source 5](#))
- **GRPO (Group Relative Policy Optimization):** 기존 강화학습(PPO)의 메모리 비효율성을 극복한 알고리즘입니다. 하나의 질문에 여러 답변 그룹을 생성하고, 그룹 내 상대 평가를 통해 더 나은 답변을 생성하도록 스스로 정책을 업데이트합니다. ([Source 8, 10](#))

GRPO: Self-Improvement Cycle



딥시크-R1의 가장 큰 기술적 특징은 지도 학습(Supervised Fine-Tuning, SFT)에 대한 의존도를 최소화하고, 강화학습(Reinforcement Learning, RL)을 통해 추론 능력을 극대화했다는 점이다.

#### 2.2.1 DeepSeek-R1-Zero와 창발적 행동

초기 모델인 **DeepSeek-R1-Zero**는 SFT 없이 기저 모델(DeepSeek-V3-Base)에 직접 강화학습을 적용하여 개발되었다. 이 과정에서 모델은 수만 번의 자체적인 문제 풀이와 검증을 반복하며 스스로 오류를 수정하고 논리를 전개하는 법을 학습했다. 흥미로운 점은 이 과정에서 '자기 검증(Self-verification)', '반성(Reflection)', '긴 사고 사슬(Long Chain of Thought)'과 같은 고도의 지능적 행동이 별도의 지도 없이 자연스럽게 창발(Emergent)되었다는 것이다.

#### 2.2.2 GRPO (Group Relative Policy Optimization)

딥시크 연구진은 기존의 PPO(Proximal Policy Optimization) 방식이 가진 메모리 비효율성을 극복하기 위해 **GRPO** 알고리즘을 도입했다. PPO는 가치 함수(Value Function)를 추정하기 위해 별도의 비평가(Critic) 모델이 필요하여 메모리 사용량이 두 배로 늘어나는 단점이 있었다. 반면, GRPO는 그룹 점수(Group Scores)로부터 베이스라인을 추정하여 비평가 모델 없이도 효율적인 학습이 가능하다.

GRPO의 핵심 메커니즘은 다음과 같다:

1. **그룹 샘플링(Group Sampling):** 하나의 질문에 대해 여러 개의 출력을 생성한다.
2. **보상 평가(Reward Evaluation):** 각 출력에 대해 정답 여부(정확성)와 포맷 준수 여부 등을 평가하여 점수를 매긴다.
3. **이점 계산(Advantage Calculation):** 그룹 내 평균 점수를 기준으로 각 출력의 상대적 우수성을 계산한다.
4. **정책 업데이트(Policy Update):** 상대적으로 우수한 출력을 생성할 확률을 높이는 방향으로 모델을 업데이트한다.

### 2.3 콜드 스타트(Cold Start)와 다단계 학습 파이프라인

DeepSeek-R1-Zero는 강력한 성능을 보였으나, 답변의 가독성이 떨어지거나 여러 언어를 혼용하는 문제(Language Mixing)가 발생했다. 이를 해결하기 위해 딥시크 팀은 다음과 같은 다단계 파이프라인을 구축하여 최종적인 **DeepSeek-R1** 모델을 완성했다.

1. **콜드 스타트(Cold Start):** 소량의 고품질 사고 사슬(CoT) 데이터를 사용하여 기저 모델을 미세 조정(Fine-tuning)한다. 이는 모델에게 '생각하는 방법'과 '가독성 있는 출력 형식'을 사전에 인지시키는 단계이다.
2. **추론 강화학습(Reasoning RL):** 콜드 스타트된 모델에 GRPO를 적용하여 수학, 코딩 등 논리적 추론 능력을 집중적으로 강화한다.
3. **거부 샘플링(Rejection Sampling) 및 SFT:** 강화학습된 모델이 생성한 데이터 중 우수한 것만을 선별하고, 여기에 일반적인 대화 데이터를 추가하여 모델을 재학습시킨다.
4. **전체 강화학습(RL for all scenarios):** 마지막으로 유용성(Helpfulness)과 무해성(Harmlessness)을 고려한 강화학습을 수행하여 모델을 인간의 선호에 맞게 정렬(Alignment)한다.

3. 성능 벤치마크 및 GPT-4o와의 비교 분석

3.1 주요 성능 지표 비교

딥시크-R1은 수학 및 코딩 영역에서 압도적인 성능을 보여주며, 이는 고가의 상용 모델과 대등한 수준이다.

| 벤치마크 항목    | 평가 내용       | DeepSeek-R1     | GPT-4o | 비고                 |
|------------|-------------|-----------------|--------|--------------------|
| MATH-500   | 수학 문제 해결 능력 | 97.3%           | 76.6%  | R1의 압도적 우위         |
| AIME 2024  | 미국 수학 경시대회  | 79.8%           | 9.3%   | 복잡한 추론에서 R1의 강점 확인 |
| MMLU       | 다중 작업 언어 이해 | 90.8%           | 88.7%  | 일반 지식 및 이해도        |
| Codeforces | 알고리즘 코딩 테스트 | 96.3 percentile | -      | 코딩 전문가 수준의 능력      |

이러한 결과는 딥시크-R1이 단순한 지식 검색을 넘어, 복잡한 문제를 단계적으로 분해하고 해결하는 능력에서 상용 SOTA(State-of-the-Art) 모델들을 위협하고 있음을 시사한다.

3.2 비용 및 경제성 분석

딥시크-R1은 성능뿐만 아니라 비용 효율성 측면에서도 혁신적이다. 특히 API를 사용할 경우의 가격 격차는 매우 크다.

- **입력 토큰 비용:** 딥시크-R1은 \$0.55/1M 토큰으로, GPT-4o(\$2.50/1M) 대비 **4.5 배 저렴**하다.
- **출력 토큰 비용:** 딥시크-R1은 \$2.19/1M 토큰으로, GPT-4o(\$10.00/1M) 대비 **4.6 배 저렴**하다.
- **로컬 구동 시:** 초기 하드웨어 투자 비용(GPU 등)을 제외하면 토큰당 한계 비용은 **\*\*0 원\*\***에 수렴한다. 이는 대량의 문서를 처리해야 하는 기업형 RAG 시스템에서 막대한 운영 비용 절감 효과를 가져온다.

3.3 로컬 구동을 위한 하드웨어 요구사항

딥시크-R1은 다양한 크기의 증류(Distilled) 모델을 제공하여 사용자의 하드웨어 환경에 맞는 최적의

선택지를 제공한다. 본 강의에서 주로 다루는 14b 모델은 일반적인 고사양 PC 에서 구동 가능한 최적의 밸런스 모델이다.

## 당신의 AI 워크스테이션: 최적의 로컬 환경 구축 가이드

DeepSeek-R1은 다양한 하드웨어 환경에 맞춰 종류(Distilled)된 모델을 제공합니다. 본 강의에서는 성능과 접근성의 균형이 가장 뛰어난 **14B** 모델을 권장합니다.

| 모델명                                 | 파라미터       | 권장 VRAM      | 최소 GPU 사양 예시              | 권장 용도                 |
|-------------------------------------|------------|--------------|---------------------------|-----------------------|
| DeepSeek-R1-Distill-Llama-8B        | 8B         | ~6GB         | RTX 3060, M1 Pro          | 일반 PC, 범용 개발          |
| <b>DeepSeek-R1-Distill-Qwen-14B</b> | <b>14B</b> | <b>~10GB</b> | <b>RTX 3080, RTX 4070</b> | <b>고성능 추론 (강의 권장)</b> |
| DeepSeek-R1-Distill-Qwen-32B        | 32B        | ~20GB        | RTX 3090, RTX 4090        | 전문가용, 복잡한 연구          |
| DeepSeek-R1 (Original)              | 671B       | ~1.3TB       | A100 80GB x 16            | 엔터프라이즈 데이터센터          |

VRAM이 부족할 경우, 모델의 일부가 CPU로 오프로드되어 속도가 저하될 수 있으므로 사양에 맞는 모델 선택이 중요합니다. (Source 12)

## 4. 로컬 실습 환경 구축: 올라마(Ollama) 활용 가이드

### 단 3단계, 10분 만에 시작하는 나만의 AI

복잡한 설정은 끝났습니다. 오픈소스 프레임워크 올라마(Ollama)를 사용하면, 마치 도커(Docker)처럼 단 하나의 명령어로 모델을 설치하고 실행할 수 있습니다.

#### 1 올라마 설치 (Install Ollama)



`ollama.com`에서 운영체제에 맞는 설치 파일 다운로드 및 실행.  
(Source 1)

#### 2 모델 실행 (Run Model)



터미널을 열고 아래 명령어 입력.  
약 9GB의 모델 파일이 자동으로  
다운로드 및 설치됩니다. (Source 1)

#### 3 대화 시작 (Start Chatting)



설치가 완료되면 `>>>` 프롬프트가  
나타납니다. 이제 당신의 로컬 AI와  
대화를 시작할 수 있습니다.

```
ollama run deepseek-r1:14b
```

### 4.1 올라마(Ollama)의 역할과 아키텍처

로컬 LLM 을 구동하기 위해서는 모델의 가중치 파일을 메모리에 로드하고, 복잡한 연산을 GPU 로 처리해주는 추론 엔진이 필요하다. 올라마(Ollama)는 이러한 복잡한 과정을 추상화하여, 마치 도커(Docker)를 사용하는 것처럼 간단한 명령어만으로 모델을 설치하고 실행할 수 있게 해주는 오픈소스 프레임워크이다.

올라마의 주요 기능은 다음과 같다:

- **모델 관리:** 모델의 다운로드, 버전 관리, 삭제 등을 CLI(Command Line Interface)로 지원한다.
- **하드웨어 가속:** 사용자의 시스템 환경(NVIDIA GPU, AMD GPU, Apple Silicon 등)을 자동으로 감지하여 최적의 하드웨어 가속을 지원한다.
- **API 서버:** 백그라운드에서 REST API 서버를 구동하여, 파이썬이나 다른 애플리케이션에서 HTTP 요청을 통해 모델과 통신할 수 있게 한다.

### 4.2 설치 및 모델 구동 실습

#### 4.2.1 올라마 설치

1. **다운로드:** 공식 웹사이트(<https://ollama.com>)에 접속하여 운영체제(Windows, macOS, Linux)에 맞는 설치 파일을 다운로드한다.

2. **설치:** 다운로드한 파일을 실행하여 설치를 진행한다. Windows 의 경우 설치 완료 후 작업 표시줄 트레이에 올라가 아이콘이 생성된다.
3. **검증:** 터미널(CMD, PowerShell, iTerm)을 열고 `ollama --version` 을 입력하여 설치가 정상적으로 되었는지 확인한다.

#### 4.2.2 딥시크-R1 모델 실행

터미널에서 다음 명령어를 입력하여 모델을 다운로드하고 실행한다. 본 강의에서는 성능과 리소스의 균형이 좋은 **14b 모델**을 사용한다.

---

```
ollama run deepseek-r1:14b
```

---

명령어를 실행하면 다음과 같은 과정이 진행된다:

1. **Pulling manifest:** 모델의 메타데이터를 확인한다.
2. **Downloading layers:** 약 9GB(14b 모델 기준)의 모델 가중치 파일을 다운로드한다. 네트워크 속도에 따라 수 분이 소요될 수 있다.
3. **Verifying checksum:** 다운로드된 파일의 무결성을 검증한다.
4. **Success & Run:** 설치가 완료되면 대화형 프롬프트(>>>)가 나타나며 즉시 대화가 가능하다.

#### 4.3 CLI 기반 대화 및 사고 과정(CoT) 확인

설치된 모델과 터미널에서 직접 대화를 나누며 딥시크-R1 의 특징인 사고 과정을 관찰할 수 있다.

---

사용자 입력:

Python 으로 피보나치 수열을 구하는 함수를 작성해줘.

모델 출력 예시:

<think>

사용자가 피보나치 수열을 구하는 Python 함수를 요청했다.

1. 재귀적 방법(Recursive)은 코드는 간단하지만 성능이 느리다.
2. 반복문(Iterative)을 사용하는 방법이 효율적이다.
3. 제너레이터(Generator)를 사용하는 방법도 메모리 효율적이다.

사용자의 의도를 고려하여 가장 일반적인 반복문 방식과 제너레이터 방식을 함께 제시하는 것이 좋겠다.

먼저 기본 개념을 설명하고 코드를 작성하자.

</think>

피보나치 수열을 구하는 방법은 여러 가지가 있습니다. 효율성을 고려하여 반복문을 사용하는 방법과... (후략)

이처럼 <think>와 </think> 태그 사이에 모델의 추론 과정이 실시간으로 출력된다. 사용자는 이 과정을 통해 모델이 어떤 논리를 통해 답변을 도출했는지 파악할 수 있으며, 이는 AI의 환각(Hallucination) 현상을 식별하는 데에도 큰 도움이 된다.

---

## 5. 랭체인(LangChain) 통합 및 애플리케이션 개발

### 단 세 줄의 코드로, 당신의 앱이 로컬 AI를 품게 됩니다

랭체인(LangChain) 프레임워크를 사용하면 기존 OpenAI API 기반 코드를 거의 그대로 재사용할 수 있습니다. 모델을 초기화하는 부분만 **ChatOllama**로 교체하면 됩니다.

#### 기존 방식 (The Old Way: Cloud API)

```
# from openai import OpenAI
#
# llm = OpenAI(
#     api_key="sk-...",
#     model="gpt-4o"
# )
#
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.6
)
```

#### 새로운 방식 (The New Way: Local AI)

```
# 3줄만 바꾸면 됩니다
# No API Key needed

from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="deepseek-r1:14b",
    temperature=0.6
)
```

최소한의 코드 변경으로  
완전한 로컬 전환

단순한 CLI 대화를 넘어, 실제 애플리케이션에 로컬 모델을 통합하기 위해 **랭체인(LangChain)** 프레임워크를 사용한다. 랭체인은 LLM 을 활용한 애플리케이션 개발을 위한 표준 프레임워크로, 모델 교체, 프롬프트 관리, 메모리 기능 등을 손쉽게 구현할 수 있게 해준다.

#### 5.1 개발 환경 설정 및 라이브러리 설치

파이썬(Python) 환경에서 다음 라이브러리들을 설치한다. 최신 랭체인 업데이트를 반영하여 langchain-ollama 패키지를 사용하는 것이 중요하다.

---

```
pip install langchain-ollama langchain-core langchain-community
```

---

#### 5.2 기본 챗봇 구현 (Context 유지)

기존의 OpenAI API 를 사용하는 코드 구조를 거의 그대로 유지하면서, 모델 인스턴스만 ChatOllama 로 교체하여 로컬 챗봇을 구현할 수 있다.

### 코드 구현 (deepseek\_simple\_chatbot.py):

---

```
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# 모델 초기화: 로컬 서버(localhost:11434)에 연결
# 컴퓨터 사양에 따라 'deepseek-r1:8b' 또는 'deepseek-r1:14b' 선택
llm = ChatOllama(model="deepseek-r1:14b", temperature=0.6)

# 시스템 메시지 설정: 모델의 페르소나 및 제약 조건 부여
messages =

print("DeepSeek-R1 챗봇이 시작되었습니다. 종료하려면 'exit'를 입력하세요.")

while True:
    user_input = input("User: ").strip()
    if user_input.lower() in ["exit", "quit", "q"]:
        print("Goodbye!")
        break

    messages.append(HumanMessage(user_input))

    # 모델 호출 (invoke)
    response = llm.invoke(messages)

    print(f"Bot: {response.content}")

    # 대화 문맥 유지를 위해 AI의 응답을 기록에 추가
    messages.append(AIMessage(response.content))
```

---

위 코드는 사용자의 입력을 누적하여 전달함으로써 멀티턴(Multi-turn) 대화가 가능하게 한다. SystemMessage 를 통해 "한국어로 답변하라"는 지침을 줌으로써 딥시크 모델의 언어 혼용 문제를 완화할 수 있다.

### 5.3 스트리밍(Streaming) 출력 구현과 사고 과정 처리

LLM의 응답 속도는 생성되는 텍스트의 길이에 비례하여 지연시간이 발생한다. 딥시크-R1은 사고 과정(<think>) 때문에 전체 텍스트가 매우 길어질 수 있어, 사용자가 응답이 멈춘 것으로 오해할 수 있다. 따라서 토큰이 생성되는 즉시 화면에 출력하는 스트리밍 방식이 필수적이다.

## 스트리밍 구현 코드:

---

```
#... 이전 설정 동일...
```

```
# invoke 대신 stream 메서드 사용
```

```
response_stream = llm.stream(messages)
```

```
full_response = ""
```

```
print("Bot: ", end="")
```

```
for chunk in response_stream:
```

```
    token = chunk.content
```

```
    print(token, end="", flush=True) # 실시간 출력
```

```
    full_response += token
```

```
print() # 줄바꿈
```

```
# 대화 기록 저장 시 사고 과정(<think>...</think>) 제거 처리 (선택 사항)
```

```
# 사고 과정이 대화 기록(Context Window)을 불필요하게 차지하는 것을 방지
```

```
if "</think>" in full_response:
```

```
    clean_response = full_response.split("</think>")[-1].strip()
```

```
else:
```

```
    clean_response = full_response
```

```
messages.append(AIMessage(clean_response))
```

---

이 코드는 stream 메서드를 사용하여 응답을 청크(Chunk) 단위로 받아 실시간으로 출력한다. 또한, 대화 기록(messages)에 저장할 때는 <think> 태그 내부의 내용을 제거함으로써, 다음 대화 턴에서 모델이 불필요하게 긴 컨텍스트를 처리해야 하는 부담을 줄여준다.

## 6. RAG(검색 증강 생성) 시스템 구축: 데이터 보안과 추론의 결합

기업이나 개인이 로컬 LLM 을 도입하는 가장 큰 목적 중 하나는 내부 문서를 안전하게 검색하고 활용하는 **RAG(Retrieval-Augmented Generation)** 시스템 구축이다.

### 6.1 RAG 아키텍처 및 데이터 프라이버시 전략

RAG 는 LLM 이 사전에 학습하지 않은 외부 데이터(PDF, 사내 매뉴얼 등)를 참조하여 답변을 생성하는 기술이다. 로컬 환경에서의 RAG 구현 시 데이터 프라이버시 수준에 따라 두 가지 전략을 취할 수 있다.

1. **하이브리드 전략:** LLM 은 로컬 디스크를 사용하되, 문서의 의미를 벡터로 변환하는 임베딩(Embedding) 과정은 OpenAI 의 고성능 모델(text-embedding-3-large 등)을 사용한다. 문서 내용이 임베딩을 위해 외부로 전송되므로 완전한 보안은 아니다.
2. **완전 로컬 전략:** 임베딩 모델까지 로컬(Ollama 의 nomic-embed-text 등)에서 구동한다. 데이터가 외부로 전혀 나가지 않는 완벽한 보안을 달성할 수 있다.

### 6.2 Streamlit 기반 RAG 애플리케이션 구현

본 실습에서는 웹 인터페이스 라이브러리인 Streamlit 을 사용하여 PDF 문서를 업로드하고 질의응답을 수행하는 RAG 앱을 구축한다.

#### 6.2.1 주요 구성 요소 (retriever.py 수정)

기존의 OpenAI 기반 코드를 로컬 모델용으로 수정한다.

---

```
# retriever.py
from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_chroma import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. 임베딩 모델 설정 (완전 로컬 전략)
# ollama pull nomic-embed-text 명령어로 사전 설치 필요
embeddings = OllamaEmbeddings(model="nomic-embed-text")

# 2. 벡터 저장소(Vector Store) 생성 함수
def create_retriever(files):
    # 문서 로드 및 청크 분할 (Chunking)
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    splits = text_splitter.split_documents(files)
```

```
# 벡터 저장 및 검색기(Retriever) 반환
vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)
return vectorstore.as_retriever()
```

---

### 6.2.2 RAG 애플리케이션 통합 (rag\_deepseek.py)

---

```
# rag_deepseek.py
import streamlit as st
from langchain_ollama import ChatOllama
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate
```

```
# 모델 초기화
```

```
llm = ChatOllama(model="deepseek-r1:14b")
```

```
st.title("DeepSeek-R1 로컬 보안 RAG")
```

```
# 프롬프트 템플릿: 사고 과정 유도를 위한 지시 포함
```

```
prompt = ChatPromptTemplate.from_template("""
```

```
주어진 컨텍스트를 바탕으로 질문에 답하세요.
```

```
<context>
```

```
{context}
```

```
</context>
```

```
질문: {input}
```

```
""")
```

```
# 체인 생성
```

```
question_answer_chain = create_stuff_documents_chain(llm, prompt)
```

```
#... (파일 업로드 및 retriever 생성 로직은 생략, 위 retriever.py 활용)...
```

```
if user_input:
```

```
    rag_chain = create_retrieval_chain(retriever, question_answer_chain)
```

```
    response = rag_chain.invoke({"input": user_input})
```

```
    st.write(response["answer"])
```

---

### 6.3 딥시크-R1 기반 RAG의 특징과 인사이트

---

딥시크-R1을 RAG에 적용하면 단순한 정보 검색을 넘어선 심층적인 분석이 가능하다. 예를 들어

"서울시 환경 정책 문서"를 기반으로 질문할 때, 기존 모델이 문서 내용을 단순히 요약하여

전달한다면, 딥시크-R1은 <think> 과정에서 정책 간의 상충 관계나 잠재적인 문제점을 논리적으로

추론하여 답변에 반영할 수 있다. 이는 단순 검색 엔진(Search Engine)을 넘어선 '인사이트 엔진(Insight Engine)'으로서의 가능성을 보여준다.

## 7. 한계점 극복 및 고급 문제 해결 (Troubleshooting)

### 현실의 벽을 넘어서: 고급 문제 해결 및 최적화 전략

DeepSeek-R1은 강력하지만, 실제 애플리케이션에 적용 시 몇 가지 기술적 난관에 부딪힐 수 있습니다. 다음은 주요 문제와 해결 방안입니다.

| 과제 (Challenge)                                                                   | 해결 방안 (Solution)                                                                                                                               |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>불안정한 도구 호출 (Tool Calling)</b><br>정형화된 JSON 출력 대신 일반 텍스트로 응답하는 경향. (Source 13) | <b>1. 프롬프트 엔지니어링:</b> 시스템 프롬프트에 JSON 포맷을 명시적으로 강제.<br><b>2. 모델 오케스트레이션:</b> 추론은 DeepSeek, 도구 호출 결정은 Llama3.1 등 다른 모델에 위임. (Source 1)           |
| <b>언어 혼용 (Language Mixing)</b><br>한국어 질문에 영어/중국어가 섞여 나오는 현상. (Source 1)          | <b>1. 시스템 프롬프트:</b> "반드시 한국어로 답변하십시오" 제약 조건 명시.<br><b>2. 온도(Temperature) 조절:</b> `temperature=0.3` 등으로 값을 낮춰 창의성을 제한하고 지시 이행력을 높임. (Source 16) |

로컬 LLM 구축 과정에서 개발자들이 빈번하게 마주치는 기술적 난관과 그 해결책을 심층적으로 다룬다.

#### 7.1 도구 호출(Tool Calling) 오류와 해결 방안

가장 치명적인 기술적 제약 사항은 **도구 호출(Tool Calling/Function Calling)의 불안정성**이다.

딥시크-R1은 강화학습 과정에서 추론 능력 향상에 집중했기 때문에, 정형화된 JSON 출력을 생성하거나 외부 API를 호출하는 훈련이 상대적으로 부족하다.

- **증상:** 랭체인(bind\_tools)을 사용하면 "status code: 400" 에러가 발생하거나, 도구를 호출하는 JSON 대신 일반 텍스트로 응답하는 현상이 발생한다.
- **원인:** Ollama의 템플릿과 딥시크-R1의 내부 포맷 간의 불일치, 또는 모델 자체의 기능 미지원.
- **해결 방안 1 (프롬프트 엔지니어링):** 도구 호출을 위한 명시적인 지침을 시스템 프롬프트에 포함시킨다. "도구를 사용해야 할 경우, 반드시 다음과 같은 JSON 포맷으로만 출력하십시오"라고 강제한다.
- **해결 방안 2 (구조화된 출력 강제):** OllamaFunctions와 같은 래퍼(Wrapper)를 사용하거나, Pydantic 파서를 통해 출력을 강제로 JSON으로 변환하여 파싱한다.
- **해결 방안 3 (모델 교체):** 도구 호출이 필수적인 에이전트(Agent) 기능을 구현해야 한다면, 추론은 딥시크에게 맡기되 도구 실행 결정은 llama3.1이나 qwen2.5와 같이 도구 호출이 잘

훈련된 모델에게 위임하는 '모델 오케스트레이션' 방식을 고려해야 한다.

## 7.2 언어 혼용(Language Mixing) 문제

딥시크-R1 은 학습 데이터의 특성상 한국어 질문에 대해 생각(<think>)은 영어로 하고, 답변은 중국어가 섞여 나오는 경우가 있다.

- **해결 방안:** 시스템 프롬프트(System Prompt)에 **\*\*\*반드시 한국어로 답변하시오(Always respond in Korean)\*\*\***라는 제약 조건을 명확히 명시한다. 또한, temperature 값을 낮춰(0.3~0.5) 모델의 창의성을 제한하고 지시 이행력을 높이는 것이 유효하다.

## 7.3 하드웨어 리소스 최적화 (OOM 방지)

로컬 LLM 은 시스템 메모리(RAM)와 비디오 메모리(VRAM)를 많이 소모한다. VRAM 이 부족하면 모델의 일부 레이어가 CPU 로 오프로드(Offload)되어 추론 속도가 급격히 느려지거나 시스템이 멈출 수 있다.

- **전략:** 자신의 GPU VRAM 용량에 맞춰 적절한 양자화(Quantization) 모델을 선택해야 한다.
  - **VRAM 8GB:** 7b 또는 8b 모델 (q4\_k\_m 양자화)
  - **VRAM 12GB:** 14b 모델 (q4\_k\_m)
  - **VRAM 24GB:** 32b 모델.
- **팁:** ollama run 실행 시 다른 GPU 사용 프로그램(게임, 그래픽 작업 등)을 종료하여 가용 VRAM 을 확보해야 한다.